1 SEQ BirdDex Cyberdeck — Project Task Sheet

Project: SEQ BirdDex
Purpose: Build an offline, battery-powered bird-identification cyberdeck for South-East Queensland and adjacent inland/coastal regions.
Target trip: September Lamington field use.
Current status: Concept / planning.
Primary design constraint: Field-useful by September, not research-perfect.

1.1 1. Executive summary

The SEQ BirdDex is a handheld or shoulder-carried cyberdeck that can identify birds from field images without internet access. It should accept photos from a Sony A7R V, take its own close-range photos, crop the bird subject automatically, classify the likely bird species, display a Pokédex-style species card, and log the sighting with time, GPS, local weather, image path, crop path, prediction confidence, and user confirmation.

The correct September target is not a perfect all-Australia classifier. The correct target is a robust offline system that works well for common and likely species in the defined region of interest, provides top-5 suggestions, handles uncertainty honestly, and creates a useful field log.

The recommended v1 stack is:

```
Raspberry Pi 5 16GB
+ Raspberry Pi AI HAT+ 26 TOPS or 13 TOPS
+ NVMe SSD
+ daylight-readable touchscreen
+ local Wi-Fi access point
+ FTP ingest from Sony A7R V
+ local camera for close-range photos
+ GPS + temperature/humidity/pressure sensor
+ detector + classifier + geotemporal re-ranker
+ SQLite sighting database
```

The device should store the trained model, labels, species cards, regional priors, reference thumbnails, and logs. It should not store the full training image dataset unless a retrieval-based fallback is added later.

1.2 2. Region of interest

The initial region of interest is the user-drawn SEQ / southern Queensland region:

```
North: Bundaberg / Agnes Water
East: Fraser Coast, Sunshine Coast, Brisbane, Gold Coast, Tweed coast
South: Tweed Heads / northern NSW edge / possible Byron-Lismore buffer
West: Goondiwindi / Miles / Tara / Dalby corridor
Central: Gympie, Maryborough, Toowoomba-region hinterland, Scenic Rim, Lamington
```

The uploaded ROI image is a concept sketch, not yet a machine-readable boundary. The first technical task is to convert the red boundary into a GeoJSON polygon that can be used for API queries and occurrence filtering.

1.2.1 2.1 Approximate ROI anchor points for initial GeoJSON draft

These are seed anchors only. They must be checked manually in QGIS or another map tool before data collection.

Anchor	Approximate location	Purpose
A	Agnes Water / Burnett coast	Northern coastal cap
B	Offshore Fraser / Hervey Bay buffer	Coastal record inclusion
C	Sunshine Coast / Noosa / Caloundra	Coastal SEQ corridor
D	Gold Coast / Tweed Heads	Southern coastal edge
E	Northern NSW buffer near Byron/Lismore	Optional overflow/vagrant buffer
F	Scenic Rim / Lamington	Core September field zone
G	Goondiwindi	South-western inland edge
H	Miles / Tara / Dalby corridor	Western dryland/inland edge
I	North Burnett / Biggenden hinterland	North-western return boundary

1.2.2 2.2 ROI outputs

Deliverables:

```
data/geo/seq_birddex_roi_v0.geojson
data/geo/seq_birddex_roi_v1_reviewed.geojson
data/geo/seq_birddex_roi_buffered_25km.geojson
reports/roi_definition.md
```

Acceptance criteria:

- The polygon can be loaded into QGIS.
- The polygon can be used in Python via `shapely` / `geopandas`.
- A 25 km buffer variant exists for vagrants and slightly out-of-bound observations.
- The ROI can filter occurrence records from ALA, eBird-derived files, and iNaturalist exports.

1.3 3. Product definition

1.3.1 3.1 User story

A user sees a bird in the field. They either photograph it with the Sony A7R V or use the cyberdeck camera. The image arrives on the cyberdeck. The cyberdeck detects the bird, crops it, predicts likely species, displays the top candidates and a readable species card, and logs the sighting offline. The user can confirm, reject, or mark the result as unsure.

1.3.2 3.2 Core functions

ID	Function	Required for September?
F-001	Take local photos from cyberdeck camera	Yes
F-002	Receive JPEGs from Sony A7R V	Yes
F-003	Watch ingest folder for new images	Yes
F-004	Detect bird subject in full-frame image	Yes
F-005	Crop bird subject	Yes
F-006	Classify cropped bird image	Yes
F-007	Display top-5 predictions	Yes
F-008	Display species card	Yes
F-009	Log timestamp, GPS, weather, image, crop, prediction	Yes
F-010	Let user confirm / reject / mark unsure	Yes
F-011	Export sightings to CSV/JSON	Yes
F-012	Run fully offline	Yes
F-013	Audio bird ID	Optional v1.5
F-014	Map view of sightings	Optional
F-015	Full Australasian species coverage	No, later
F-016	RAW file classification	No, use JPEG for v1

1.3.3 3.3 Non-functional requirements

ID	Requirement	Target
NFR-001	Offline operation	No internet required after setup
NFR-002	Battery runtime	Minimum 4 h; stretch target 8 h
NFR-003	Time to result	Target < 5 s per image; stretch < 2 s
NFR-004	Storage robustness	No database corruption after normal shutdown
NFR-005	Field UX	No terminal required during use
NFR-006	Uncertainty handling	Must allow “unknown/unsure” state
NFR-007	Image retention	Store original JPEG and crop
NFR-008	Reproducibility	Model version and dataset manifest logged
NFR-009	Maintainability	Repo must run from clean setup scripts

---

1.4 4. System architecture

1.4.1 4.1 High-level pipeline

```
Sony A7R V / cyberdeck camera / manual upload
        |
```

```
        v
Image ingest service
        |
        v
Metadata extractor
  - EXIF timestamp
  - camera model
  - focal length if present
  - GPS if present
        |
        v
Bird detector
  - finds one or more bird boxes
        |
        v
Cropper
  - crops subject
  - pads crop safely
  - stores crop image
        |
        v
Species classifier
  - returns class probabilities
        |
        v
Geotemporal prior
  - ROI occurrence
  - month/season
  - habitat/climate
  - local checklist
        |
        v
Decision layer
  - top-5
  - confidence
  - unknown threshold
        |
        v
User interface + SQLite log
```

1.4.2 4.2 Device service architecture

```
birddex-ingest.service
  Watches FTP/upload/camera folders for new images.

birddex-infer.service
  Runs detection, cropping, classification, re-ranking, and logging.
```

```
birddex-ui.service
  Runs local touchscreen/web UI.

birddex-sensors.service
  Reads GPS, pressure, temperature, humidity, optional light sensor.

birddex-backup.service
  Exports sightings and model metadata to USB/NVMe backup.
```

1.4.3 4.3 Ingest modes

Mode	Path	Notes
Sony FTP JPEG	A7R V -> Pi Wi-Fi AP -> FTP folder	Preferred field path
USB copy	Camera/card -> cyberdeck	Backup path
Pi camera	Local camera capture	Close-range or demo path
Phone upload	Phone browser -> local web UI	Optional

1.4.4 4.4 RAW vs JPEG decision

Use JPEG for real-time recognition.

Rationale:

- RAW/ARW requires demosaicing, white balance, colour conversion, and resizing before inference.
- RAW transfer is slower and wastes battery.
- Sony A7R V can keep RAW on the camera card while sending a smaller JPEG to the cyberdeck.
- For model training, RAW files can be archived later, but the field classifier should operate on JPEGs.

1.5 5. Hardware recommendation

1.5.1 5.1 Recommended September build

Subsystem	Recommended part	Reason
Compute	Raspberry Pi 5 16GB	Best ecosystem, camera support, GPIO, documentation
AI acceleration	Raspberry Pi AI HAT+ 26 TOPS	Strong edge vision acceleration with Hailo-8
Storage	256GB–1TB NVMe SSD	Fast logs/images/model storage
Screen	5–7 inch touchscreen, high brightness preferred	Field UI
Local camera	Pi Camera Module 3 / HQ Camera / AI Camera	Close-range capture and testing

Subsystem	Recommended part	Reason
External camera	Sony A7R V JPEG FTP transfer	High-quality bird photos
GPS	u-blox USB/serial GPS	Location/time logging
Weather	BME280/BME680/SHT31	Temperature/humidity/pressure
Power	USB-C PD battery bank or Li-ion pack with BMS	Field runtime
Controls	Physical buttons + touchscreen	Glove/sunlight-friendly UI
Enclosure	3D-printed cyberdeck shell	Custom ergonomics and mounting

1.5.2 5.2 Hardware alternatives

Platform	Strengths	Weaknesses	Recommendation
Pi 5 + AI HAT+	Low software risk, strong ecosystem, Hailo acceleration, good camera/GPIO support	Model export to Hailo can be annoying	Best v1 platform
Jetson Orin Nano Super	Much stronger ML flexibility, CUDA/TensorRT, 67 TOPS class device	Higher power, more heat, less handheld-friendly	Best high-performance dev platform
Orange Pi 5 Plus / RK3588	Strong SBC CPU/NPU specs	NPU software friction, weaker ecosystem	Avoid for deadline unless experimenting
Laptop/tablet backend	Very flexible	Not cyberdeck/off-grid elegant	Useful for early dev only
Phone app only	Best finished bird ID experience	Not custom/offline cyberdeck	Use as validation, not core build

1.5.3 5.3 Power budget target

Approximate v1 power planning values:

Component	Expected draw class	Notes
Pi 5	Medium	Depends heavily on CPU load, screen, peripherals
AI HAT+	Low to medium	Accelerator is efficient, but full system still matters
Screen	Medium to high	Often the dominant field drain
NVMe	Low to medium	Spikes during writes
GPS/weather	Low	Negligible relative to screen/compute
Wi-Fi AP	Low to medium	Required for A7R ingest

Component	Expected draw class	Notes
Camera	Low to medium	Depends on module and use

Runtime design rules:

- Screen timeout after 30–60 s.
- Classify on image arrival or button press, not continuous heavy inference.
- Do not run large transformer inference continuously.
- Use a graceful shutdown button.
- Log battery voltage if using a custom pack.

1.5.4 5.4 Hardware decision gate

Choose Pi 5 + AI HAT+ if:

- September field reliability matters more than peak ML speed.
- You want Raspberry Pi camera/GPIO/support to work cleanly.
- You are willing to tune/export models to Hailo or keep an ONNX/TFLite fallback.

Choose Jetson Orin Nano Super if:

- You want to run larger YOLO/ConvNeXt/ViT models locally.
- You accept 7–25 W class power behaviour and extra thermal design.
- You want CUDA/TensorRT more than Pi ecosystem convenience.

1.6 6. Dataset plan

1.6.1 6.1 Dataset principle

For v1, use supervised labelled data. Do not start with SimCLR as the main model path.

Reason:

- A species classifier needs species labels.
- Self-supervised learning can improve representations later, but it does not create species names by itself.
- The fastest reliable path is transfer learning from pretrained image models, fine-tuned on labelled Australian/SEQ bird images.

1.6.2 6.2 Dataset sources

Source	Use	Notes
iNaturalist Australia	Labelled bird images, metadata, observations	Track image license and attribution
Atlas of Living Australia	Occurrence data, taxonomy, species profiles, spatial filtering	Good for ROI occurrence filtering
eBird / EBD / Status and Trends	Occurrence, seasonality, checklists, relative abundance	Strong for geotemporal priors

Source	Use	Notes
BirdLife/Birdata	Australian bird context and occurrence records	Useful for local validity checks
User A7R V photos	Domain adaptation and validation	Most valuable after field trips

1.6.3 6.3 Licensing requirements

Every downloaded image must store:

```
source
observation_id
photo_id
common_name
scientific_name
taxon_id
license
creator/attribution
source_url
date_observed
latitude/longitude if available and non-sensitive
quality_grade
```

Do not train on or redistribute images unless the license permits the intended use. For personal experiments this is lower risk, but the manifest should still preserve attribution and license metadata.

1.6.4 6.4 Initial species scope

Use tiers.

Tier	Scope	Target use
Tier 1	100–200 common/likely ROI species	September MVP
Tier 2	300–500 broader QLD / northern NSW species	Post-MVP expansion
Tier 3	700+ Australian regular species	Long-term goal
Tier 4	Vagrants / rare records / subspecies / sex-age morphs	Research-grade extension

1.6.5 6.5 Dataset quality rules

- Split by observation/user/date where possible, not naive random image split.
- Remove near-duplicates across train/validation/test.
- Prefer images where bird is visible and labelled to species.
- Keep difficult images for validation, not only pretty field-guide shots.
- Track class imbalance.
- Group visually similar species for confusion analysis.

1.6.6 6.6 Data deliverables

```
data/manifests/roi_species_candidates.parquet
data/manifests/species_priority_tiers.csv
data/manifests/images_manifest.parquet
data/splits/train.csv
data/splits/val.csv
data/splits/test.csv
data/reports/license_report.md
data/reports/class_balance_report.md
data/reports/duplicate_audit.md
```

1.7 7. Model plan

1.7.1 7.1 Recommended v1 model architecture

```
Detector:
  YOLO-style lightweight bird detector
  Purpose: find birds in raw photos

Classifier:
  EfficientNetV2-B0/B1 or MobileNetV3-Large
  Purpose: classify cropped bird image

Re-ranker:
  Visual probability + location prior + month prior + habitat prior

Decision layer:
  Top-5 display + confidence thresholds + unknown state
```

1.7.2 7.2 Detector task

The detector is not required to identify species. It only needs to find bird-shaped objects.

Inputs:

```
Full-resolution or resized JPEG
```

Outputs:

```
bounding boxes
box confidence
crop image paths
```

Acceptance criteria:

- Detects birds in field images where the bird is small but visible.
- Handles branches, sky, water, grass, and clutter.
- Can return multiple birds.
- Does not crop so tightly that tail/beak/wings are lost.

1.7.3 7.3 Classifier task

Inputs:

```
Bird crop, usually 224-384 px input resolution
```

Outputs:

```
species probabilities
top-5 species
confidence / entropy / uncertainty
```

Candidate v1 backbones:

Backbone	Strength	Weakness
MobileNetV3-Large	Very efficient, mobile-friendly	May underperform on fine-grained species
EfficientNetV2-B0/B1	Good accuracy/speed compromise	Export and quantisation need care
ConvNeXt-Tiny	Stronger visual features	Heavier for Pi
ViT/Swin	Strong with enough data/compute	Not ideal for Pi v1
BioCLIP-style embedding	Useful for biological similarity and retrieval	Heavier and more complex deployment

1.7.4 7.4 Geotemporal re-ranking

The model should not rely on visual score alone. The system should re-rank predictions using local probability.

Example scoring:

```
final_score(species) =
  visual_logit_score
  + alpha * roi_occurrence_prior
  + beta  * month_prior
  + gamma * habitat_prior
  + delta * user_history_prior
```

Rules:

- Do not remove rare/vagrant species completely.
- Penalise unlikely species rather than making them impossible.
- Provide a "vagrant/rare but possible" warning when applicable.
- Use a toggle for "strict local mode" versus "wide/vagrant mode".

1.7.5 7.5 Unknown and uncertainty logic

The device must be allowed to say "not sure." Suggested rules:

```
If max_probability < threshold_low:
  display Unknown / uncertain
```

```
If top1_probability - top2_probability < margin_threshold:
  display Similar species warning

If visual score high but geotemporal prior low:
  display Possible rare/vagrant warning

If detector crop confidence low:
  display Crop uncertain warning
```

1.7.6 7.6 Optional embedding fallback

A retrieval module may be added later.

```
Image crop -> embedding model -> nearest reference examples -> species vote
```

Device storage:

```
reference_embeddings.faiss
embedding_metadata.sqlite
reference_thumbnails/
```

Use case:

- Low-confidence classifier output.
- Showing similar reference birds.
- Adding new species with fewer examples.

Do not make this required for September unless the classifier path is already stable.

1.8 8. Device data model

1.8.1 8.1 SQLite schema

```sql
CREATE TABLE model_versions (
    id INTEGER PRIMARY KEY,
    model_name TEXT NOT NULL,
    model_type TEXT NOT NULL,
    version TEXT NOT NULL,
    file_path TEXT NOT NULL,
    created_at TEXT,
    training_manifest_hash TEXT,
    notes TEXT
);

CREATE TABLE species (
    species_id TEXT PRIMARY KEY,
    birddex_id TEXT UNIQUE NOT NULL,
    common_name TEXT NOT NULL,
    scientific_name TEXT NOT NULL,
```

```
    family TEXT,
    group_name TEXT,
    size_text TEXT,
    habitat_text TEXT,
    climate_text TEXT,
    distribution_text TEXT,
    similar_species_json TEXT,
    field_marks_text TEXT,
    facts_text TEXT,
    conservation_status TEXT,
    reference_image_path TEXT
);

CREATE TABLE observations (
    id INTEGER PRIMARY KEY,
    timestamp_local TEXT NOT NULL,
    timestamp_utc TEXT,
    latitude REAL,
    longitude REAL,
    altitude_m REAL,
    temperature_c REAL,
    humidity_pct REAL,
    pressure_hpa REAL,
    light_lux REAL,
    source TEXT,
    original_image_path TEXT,
    crop_image_path TEXT,
    detector_confidence REAL,
    top1_species_id TEXT,
    top1_common_name TEXT,
    top1_confidence REAL,
    top5_json TEXT,
    model_version_id INTEGER,
    user_confirmed_species_id TEXT,
    user_verdict TEXT,
    notes TEXT,
    FOREIGN KEY(model_version_id) REFERENCES model_versions(id)
);

CREATE TABLE field_sessions (
    id INTEGER PRIMARY KEY,
    session_name TEXT,
    start_time TEXT,
    end_time TEXT,
    location_label TEXT,
    notes TEXT
);
```

1.8.2 8.2 File layout on cyberdeck

```
/opt/birddex/
  app/
  models/
    detector.hef
    classifier.hef
    classifier_fallback.onnx
    label_map.json
  data/
    birddex.sqlite
    priors.sqlite
    taxonomy.json
  media/
    original/
    crops/
    reference/
  logs/
  exports/
  config/
    device.yaml
    thresholds.yaml
    roi.geojson
```

1.9 9. User interface requirements

1.9.1 9.1 Main screens

Screen	Required content
Home	Camera status, battery, GPS lock, latest sighting
Incoming photo	Original image, detected crop(s), processing status
Prediction	Top-5 list, confidence, warnings
Species card	Name, BirdDex ID, reference image, facts, habitat, climate
Confirm sighting	Confirm / wrong / unsure / add note
Logbook	Recent observations, search/filter
Export	CSV/JSON export, backup status
Settings	Model version, thresholds, ROI mode, storage

1.9.2 9.2 Prediction display format

Example:

```
Likely ID
BDX-0042 - Regent Bowerbird
```

```
Confidence: 0.72
Local likelihood: High

Other candidates:
2. Satin Bowerbird - 0.14
3. Green Catbird - 0.06
4. Golden Whistler - 0.03
5. Eastern Whipbird - 0.02

Warnings:
- Good crop
- Similar species possible
- Location/month plausible

Actions:
[Confirm] [Wrong] [Unsure] [Show similar]
```

1.9.3 9.3 Physical controls

Recommended controls:

Button	Function
Capture	Take local photo
Process latest	Run inference on latest received image
Confirm	Mark prediction correct
Unsure	Store for later review
Back	Return to previous screen
Power	Safe shutdown / wake

Touchscreen is useful, but physical buttons are more reliable in bright outdoor conditions.

1.10 10. Training and evaluation plan

1.10.1 10.1 Desktop training flow

```
1. Build ROI species list.
2. Download labelled images with license metadata.
3. Clean dataset.
4. Build train/val/test splits.
5. Train baseline classifier.
6. Evaluate baseline.
7. Add detector/cropper.
8. Evaluate raw-image pipeline.
9. Calibrate confidence.
10. Export model to deployment format.
11. Run Pi-side inference tests.
```

1.10.2 10.2 Metrics

Metric	Purpose	Target for MVP
Top-1 accuracy	Direct correctness	Useful but not sole metric
Top-5 accuracy	Field shortlist quality	High priority
Per-class accuracy	Find weak species	Required
Confusion matrix	Similar species clusters	Required
Expected calibration error	Confidence honesty	Required if possible
Unknown rejection precision	Avoid false certainty	Required
Detector recall	Bird found in image	High priority
Time per image	Field usability	< 5 s target
Battery runtime	Field usability	> 4 h minimum

1.10.3 10.3 Test sets

Create multiple test sets:

```
clean_test:
  curated labelled images from known sources

field_like_test:
  messy images, branches, distance, bad lighting

own_camera_test:
  A7R V photos from real field use

negative_test:
  images without birds, signs, trees, dogs, insects, mammals

similar_species_test:
  honeyeaters, robins, thornbills, bowerbirds, fairywrens, raptors
```

1.10.4 10.4 Acceptance criteria for model v1

The model is acceptable for September if:

- Top-5 results are useful for common ROI birds.
- The device can say "unsure" when confidence is low.
- Similar species are grouped sensibly.
- The cropper works on real field images.
- Inference runs offline on the chosen device.
- Every prediction logs model version and confidence.

1.11 11. Development roadmap

1.11.1 Phase 0 — Project skeleton and ROI

Tasks:

- ☐ Create Git repository.
- ☐ Add project README.
- ☐ Add `data/`, `src/`, `device/`, `models/`, `reports/`, `tests/` layout.
- ☐ Convert ROI sketch into GeoJSON.
- ☐ Add ROI visualisation script.
- ☐ Query initial species occurrence list.
- ☐ Produce Tier 1 species list.

Deliverables:

```
README.md
roi.geojson
reports/roi_definition.md
data/manifests/roi_species_candidates.csv
```

1.11.2 Phase 1 — Dataset builder

Tasks:

- ☐ Implement iNaturalist/ALA metadata collector.
- ☐ Implement image downloader with license filter.
- ☐ Store source/license/attribution metadata.
- ☐ Build duplicate detection workflow.
- ☐ Build species balance report.
- ☐ Build train/val/test splits.

Deliverables:

```
src/datasets/build_species_list.py
src/datasets/download_images.py
src/datasets/build_splits.py
data/manifests/images_manifest.parquet
data/reports/license_report.md
data/reports/class_balance_report.md
```

1.11.3 Phase 2 — Baseline classifier

Tasks:

- ☐ Train MobileNetV3 or EfficientNetV2 baseline.
- ☐ Add augmentation pipeline.
- ☐ Add class weighting or sampler for imbalance.
- ☐ Save model checkpoints.
- ☐ Generate evaluation report.
- ☐ Export model to ONNX/TFLite.

Deliverables:

```
models/classifier_v0.pt
models/classifier_v0.onnx
reports/classifier_v0_eval.md
reports/confusion_matrix_v0.png
```

1.11.4 Phase 3 — Detector and cropper

Tasks:

- ☐ Select detector model.
- ☐ Test detector on field-like bird photos.
- ☐ Implement crop padding and aspect-ratio handling.
- ☐ Store crop previews.
- ☐ Evaluate detection failures.

Deliverables:

```
models/detector_v0.onnx
src/inference/detect_and_crop.py
reports/crop_quality_report.md
```

1.11.5 Phase 4 — End-to-end desktop inference

Tasks:

- ☐ Build `predict_image.py`.
- ☐ Input full image, output crop + top-5.
- ☐ Add geotemporal re-ranking stub.
- ☐ Add JSON result export.
- ☐ Add batch inference mode for test folders.

Deliverables:

```
src/inference/predict_image.py
src/inference/rerank.py
reports/end_to_end_eval.md
```

1.11.6 Phase 5 — Cyberdeck runtime

Tasks:

- ☐ Install OS.
- ☐ Configure NVMe storage.
- ☐ Configure local Wi-Fi AP.
- ☐ Configure FTP server.
- ☐ Configure systemd services.
- ☐ Add image folder watcher.
- ☐ Run inference on new files.
- ☐ Write results to SQLite.

Deliverables:

```
device/systemd/birddex-ingest.service
device/systemd/birddex-infer.service
device/config/device.yaml
device/scripts/install_device.sh
```

1.11.7 Phase 6 — UI

Tasks:

- ☐ Build local web UI or touchscreen UI.
- ☐ Display latest image and crop.
- ☐ Display top-5 predictions.
- ☐ Display species card.
- ☐ Add confirm/wrong/unsure buttons.
- ☐ Add logbook view.
- ☐ Add export button.

Deliverables:

```
device/ui/
device/api/
reports/ui_test_report.md
```

1.11.8 Phase 7 — Sensors and logging

Tasks:

- ☐ Integrate GPS.
- ☐ Integrate weather sensor.
- ☐ Add timestamp fallback via RTC.
- ☐ Add battery status if available.
- ☐ Log sensor snapshot per observation.

Deliverables:

```
src/device/sensors.py
reports/sensor_test_report.md
```

1.11.9 Phase 8 — Field hardening

Tasks:

- ☐ Battery runtime test.
- ☐ Thermal test.
- ☐ Screen readability test.
- ☐ FTP transfer reliability test.
- ☐ Sudden power-loss test using copied database only.
- ☐ Graceful shutdown test.
- ☐ Enclosure revision.

Deliverables:

```
reports/battery_runtime.md
reports/thermal_test.md
reports/field_trial_01.md
enclosure/stl/v1/
```

1.12 12. September MVP definition

The MVP is complete when:

- ☐ Device boots into UI without terminal work.
- ☐ Device creates its own Wi-Fi AP.
- ☐ Sony A7R V can send JPEGs to the device.
- ☐ New JPEGs are processed automatically.
- ☐ The bird subject is cropped automatically.
- ☐ The classifier returns a top-5 list offline.
- ☐ The UI shows a species card.
- ☐ The user can confirm/wrong/unsure the prediction.
- ☐ GPS/weather/time/image/crop/prediction are logged.
- ☐ Logs can be exported after the trip.
- ☐ Battery runtime is at least 4 hours in realistic field use.

The MVP is not required to:

- ☐ Identify every Australian bird.
- ☐ Run large transformer models.
- ☐ Classify RAW files directly.
- ☐ Work perfectly on tiny distant birds.
- ☐ Upload to eBird/Birdata/iNaturalist automatically.
- ☐ Be waterproof.

1.13 13. Suggested repository structure

```
seq-birddex/
  README.md
  pyproject.toml
  requirements.txt
  .gitignore

  data/
    geo/
    manifests/
    splits/
    external/
    interim/
    processed/

  src/
    birddex/
      datasets/
      training/
      inference/
      priors/
      export/
```

```
    device/
    ui/
    utils/

models/
  detector/
  classifier/
  exported/

device/
  systemd/
  config/
  scripts/
  api/
  ui/

species_cards/
  cards/
  reference_images/
  species_cards.sqlite

reports/
  figures/
  eval/
  field_tests/

tests/
  test_dataset_manifest.py
  test_inference_pipeline.py
  test_sqlite_schema.py
  test_reranker.py
```

1.14 14. Risk register

Risk	Severity	Probability	Mitigation
Dataset is noisy or mislabelled	High	High	Manual audit, use research-grade sources, filter low-quality observations
Similar species confusion	High	High	Top-5 UI, similar-species cards, geotemporal priors

Risk	Severity	Probability	Mitigation
Bird too small in frame	High	Medium	Use A7R V JPEGs, detector, crop inspection, confidence gating
Hailo export friction	Medium	Medium	Keep ONNX/TFLite fallback; test export early
Battery runtime poor	Medium	Medium	Screen timeout, classify-on-demand, large battery, low-power mode
UI too fiddly outdoors	High	Medium	Physical buttons, large fonts, high contrast
Scope creep	High	High	Lock v1 to ROI + Tier 1 species
Legal/licensing issues	Medium	Medium	Store license metadata, avoid redistribution of restricted images
Corrupt logs after power loss	Medium	Low/Medium	SQLite WAL mode, graceful shutdown, periodic backups
Overconfident wrong IDs	High	Medium	Calibration, unknown threshold, confidence display

---

1.15 15. Immediate next actions

Start with these in order:

1. Convert the red ROI sketch into `roi.geojson`.
2. Generate an initial species candidate list for the ROI.
3. Select Tier 1 species for September.
4. Build a licensed image metadata manifest.
5. Train a small baseline classifier on desktop.
6. Test classifier on ugly real-world bird photos.
7. Prototype the Sony A7R V to Pi JPEG transfer path.
8. Build the Pi folder watcher and SQLite logger.
9. Add the simple UI.
10. Only then optimise model acceleration.

1.16 16. Source notes

The following sources should be checked during implementation, because hardware/API support can change:

- Raspberry Pi AI HAT+ documentation: https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html
- NVIDIA Jetson Orin Nano Super Developer Kit: https://www.nvidia.com/en-au/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/
- Sony Camera Remote SDK: https://support.d-imaging.sony.co.jp/app/sdk/en/index.html
- Sony camera/mobile compatibility support: https://www.sony.com.au/electronics/support/articles/00284526
- Atlas of Living Australia API documentation: https://docs.ala.org.au/
- eBird API documentation: https://documenter.getpostman.com/view/664302/S1ENwy59
- eBird data products: https://science.ebird.org/en/use-ebird-data/download-ebird-data-products
- iNaturalist Australia: https://inaturalist.ala.org.au/
- BirdNET: https://birdnet.cornell.edu/
- BirdNET-Pi: https://www.birdweather.com/birdnetpi
- BioCLIP 2 model card: https://huggingface.co/imageomics/bioclip-2

1.17 17. Final design stance

For this project, the device should prioritise reliability over theoretical peak model accuracy. A compact detector/classifier with good crops, good uncertainty handling, strong local species priors, and a useful logbook will outperform a large fragile model in real field use.

The first hard proof should be:

```
A real A7R V bird JPEG arrives on the cyberdeck,
the bird is cropped automatically,
the offline classifier returns a plausible top-5,
the UI shows a species card,
and the sighting is saved with GPS/weather/time metadata.
```

That is the milestone that proves the project is real.